INF1103: Lab 6

Basics of Object-Oriented Programming

1. Introduction

Welcome to Week 8!

You've survived procedural programming for the past 7 weeks, created something useful (hopefully) and presented it to the boss (lab in charge is filling this role for now!). However, you can't help but feel there must be a better way to write and organize code, and there is! Welcome to Object Oriented Programming.

In Week 5, you built an Inventory Management System using dictionaries, lists, functions and JSON persistence. Your program could add, update, search and display products, and save the inventory so that it could be used again when the program restarted.

This week, we will refactor the same system using Object-Oriented Programming (OOP). Instead of thinking primarily about the functions that manipulate our data, we will begin by asking:

"What objects exist in our system, what information do they contain, and what can they do?"

One clever way to identify objects is to describe what you are trying to do, and think about the nouns in that description, for example:

"A **robot** needs to kick a **ball** into a net and a **scoreboard** will update", the bold words are nouns, each could be represented by a class with its own attributes and methods.

For our inventory system, one obvious noun (object) is a Product.

A Product has information (attributes) such as:

- product ID
- name
- price
- stock quantity

A Product can also perform operations (methods) related to itself, such as:

- update its stock
- display its information

This now gives us enough information about the object we want to create to build a blueprint (class). We can use a very basic class diagram (discussed in lecture) for this purpose and expand it as we go :

Product product.py
productid name price stock
update_stock() display_information()

Remember encapsulation, although python doesn't enforce it, we can consider which attributes should and shouldn't be changed directly, which would be better to control through code, indeed in most languages no attribute should be controlled directly, and all should be only modified through public functions.

Also, remember your SOLID principles, specifically for this lab, Single Responsibility Principle (SRP):

Single Responsibility Principle (SRP): A class should have one clear responsibility.

For this lab, our Product class should be responsible for representing and managing a product. It should not also be responsible for loading JSON files, saving files, displaying the entire menu system, or managing the whole application. We will explore more SOLID principles in later weeks.

2. Assignment

2.1 The Scenario

The store manager is happy with the inventory system you created in Week 5.

However, the current system is based primarily on dictionaries and functions. As the system grows, it may become harder to manage the data and the operations associated with individual products.

You have been asked to refactor the system using Object-Oriented Programming.

Your program should still provide the same basic functionality as the Week 5 system, but the inventory items should now be represented as objects.

2.2 Identify the Objects

Before writing code, think about the system you built last week.

You created an **Inventory System**; the inventory system had a number of **products** which could be displayed. To access the system you had a **menu**, you could also activate a **file** save / load system to store and retrieve information.

So, I would ask you where the nouns are, but I've made it easy and put them in bold (just this once). Each of these can be considered objects which we can build classes for.

We already considered a class diagram for the product class in our introduction, but we can touch it up a bit more to consider types and encapsulation:

Product
- __product_id:String - __name:String - __price:float - __stock:int
+ update_stock(quantity : int):void + display_information():void

these 4 are to be encapsulated

public functions

2.3 Create the Product Class

From the above class diagram, you should now be able to create a class for product. Although Python doesn't force us to worry about the data type or access modifiers, it is good to still consider them from a design perspective.

We should also have our __init__ method which is our initializer (or constructor) remember, the parameters here can have the same names or different names to our attributes, but we can differentiate with 'self.'.

```
def __init__(self, product):
        self.__product = product
```

So, the product in the brackets is our parameter, a value being passed from outside, the self.__product is our attribute, declared in __init__, so self.__product = product is saying make the attribute take the value of the parameter passed to the initializer.

2.4 Encapsulation

Remember setting the stock directly from outside the class gives the Product class no control over how the value is changed. If we use product.update_stock(50) then we can decide how the value 50 is treated before internally assigning a value to self.__stock.

In Python, __attribute indicates that an attribute is intended to be treated as private. It is a convention rather than a strict access restriction. This helps establish that external code should interact with the Product through its public methods.

```
product.update_stock(50)
```

Product has update_stock. By saying product., we are calling the class

rather than directly manipulating

```
product.__stock = 50 # this won't work exactly! No errors, just silently creates new attribute
```

Learning Note: Python uses name mangling with product.__stock, so to access it you would actually have to write product._Product__stock. More for avoiding name collisions between classes and subclasses (more on this next week).

2.5 Apply the Single Responsibility Principle

Your Product class should have one clear responsibility:

Represent and manage a product.

Therefore, the Product class should not be responsible for:

- loading inventory.json
- saving inventory.json
- displaying the main menu
- asking the user for input
- managing the entire inventory

So when creating a class, always ask yourself 'what is the one responsibility I have decided for this class' and make sure any code you add to that class fits that responsibility (and don't say 'this class has the single responsibility to do everything', that ain't good coding practice!

2.6 Create an Inventory Class inventory.py

So who is responsible for managing a collection of products, from our sentence above that was the inventory system, so inventory can be our next class.

SRP: An Inventory is responsible for managing a collection of Products.

Create an Inventory class based on the concepts below, also practice drawing your own class diagram for this one.

The Inventory should contain a collection of Product objects. The code below would not be written in Inventory, but instead in your main code. main.py

```
inventory = Inventory() # creating the inventory object from the Inventory class

product1 = Product("P001", "Laptop", 1200.00, 15) # creating product from Product class, via init
product2 = Product("P002", "Mouse", 25.50, 40)

inventory.add_product(product1) # pass the product to inventory class to store in a list
inventory.add_product(product2)
```

The Inventory class should provide methods for operations such as:

```
add_product()
search_product()
display_all()
```

Notice the difference in responsibilities:

```
Product
    → manages one product

Inventory
    → manages a collection of products
```

This is an example of applying the Single Responsibility Principle.

2.7 Refactor the Week 5 Program

Take your Week 5 Inventory Management System and refactor it so that it uses your new classes.

Your program should still provide the following functionality:

1. Display all products
2. Add a product
3. Update stock
4. Search for a product
5. Save inventory
6. Exit

However, the program should now work with Product objects rather than dictionaries representing products.

```
product = Product("P004", "Monitor", 299.99, 10)

inventory.add_product(product)
```

Your menu should interact with the Inventory object rather than directly manipulating product data.

2.8 Separate the Responsibilities

Your final program should have a main program, an inventory class and a product class. This structure we will continue to improve on as you learn more OOP concepts.

The responsibilities for each class (Single Responsibility Principle) should be:

```
Product
    • Stores product information
    • Manages its own state

Inventory
    • Stores products
    • Adds products
    • Searches products
    • Displays products

Main Program (for now also creates the products and inventory)
    • Displays the menu
    • Gets user input
    • Calls the appropriate Inventory methods

File Handling
    • Loads inventory data
    • Saves inventory data
```

You do not need to create a separate file-handling class or a menu class yet, unless you really want to. Our main learning objective for this week is to start thinking about which class should be responsible for which behavior.

3. Sample Output

Your final program should provide functionality similar to the Week 5 system. The joys of making your coding infrastructure better, you do a lot of work but the output doesn't change! However, it does mean that what is going on behind the scenes has a much better software design and will allow for modularity and easier additions and changes going forward (which is good right?)

4. Git

It should go without saying at this point, but continue to use git to save your work and changes. Some suggested points where you might want to do a git commit after:

- Created Product class
- Created Inventory class
- Refactored inventory system using OOP
- Completed OOP inventory management system

Finally, push your code to your GitHub repository.

You may also have noticed that the number of files you are creating has started to increase, in object-oriented programming it is not uncommon to have hundreds of files, so doing add, commit, push for each new or modified file could get a little time consuming.

I usually do git add .
adds ALL the files in the target directory and all the child folders and all their respective contents

5. Docker

Your Docker workflow remains the same.

You do not need to rebuild the Docker image simply because you have changed your Python code when using the volume mount.

Run your program using the Docker volume mount:

```
docker run --rm -it -v "${PWD}:/app" -w /app inf1103-labs-smart-auditor
```

Your local Python files should be mounted into the container.

This means you can:

- Edit your Python code.
- Save the file.
- Run the Docker command again.
- Test your changes.

A Docker image rebuild is only required when the image itself needs to change, such as when changing the Dockerfile or adding dependencies.

6. What You Need to Submit

- Your Python file(s) containing the OOP inventory system.
- Your updated inventory.json file.
- Your UML class diagram (for all classes you created).
- Your Git log.
- `git log > log.txt`
- Screenshot showing your Docker container running successfully.
- Screenshot showing your program accepting input and producing the expected results.
- Put all required files into one folder.
- Zip the folder and upload it to xSite.

7. Reflection Questions

1. What is the difference between a class and an object?
2. Why is Product a suitable class for this system?
3. What data belongs inside a Product object?
4. Why should the Product class manage its own stock rather than the main program directly changing it?
5. What does encapsulation mean in your program?
6. What is the responsibility of the Product class?
7. What is the responsibility of the Inventory class?
8. How does separating these responsibilities demonstrate the Single Responsibility Principle?
9. What information does the main program need to know about a Product?
10. What information about the internal implementation of Product does the main program not need to know?

8. Looking Ahead

This week, we have created a basic object-oriented design using:

- Classes
- Objects
- Attributes
- Methods
- Constructors
- self
- Encapsulation
- UML class diagrams
- Single Responsibility Principle

Next week, we will extend this design.

Consider what might happen if our inventory contains different types of products, each with its own behaviour, such as having physical and digital products. How could we design our classes so that different types of products can share common functionality while also having their own behaviour?

Get ready to enter the world of coding using inheritance and abstraction!